

My Project

Generated by Doxygen 1.9.8

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 sprite_s Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 h	5
3.1.2.2 make_disappear	5
3.1.2.3 message	6
3.1.2.4 w	6
3.1.2.5 x	6
3.1.2.6 y	6
3.2 textures_s Struct Reference	6
3.2.1 Detailed Description	7
3.2.2 Member Data Documentation	7
3.2.2.1 background	7
3.2.2.2 but	7
3.2.2.3 finish	7
3.2.2.4 font	7
3.2.2.5 meteorite	7
3.2.2.6 missile	7
3.2.2.7 music	8
3.2.2.8 sound_explosion	8
3.2.2.9 sound_lose	8
3.2.2.10 sound_mini_explosion	8
3.2.2.11 sound_mini_win	8
3.2.2.12 sound_win	8
3.2.2.13 spaceship	8
3.3 world_s Struct Reference	9
3.3.1 Detailed Description	9
3.3.2 Member Data Documentation	9
3.3.2.1 but	9
3.3.2.2 collision_but	9
3.3.2.3 compteur_missile_max	9
3.3.2.4 debut_temps	10
3.3.2.5 finish_line	10
3.3.2.6 gameover	10
3.3.2.7 message_final	10
3.3.2.8 missile	10

3.3.2.9 sprite	10
3.3.2.10 stop	10
3.3.2.11 temps_final	10
3.3.2.12 vy	11
3.3.2.13 walls	11
4 File Documentation	13
4.1 constantes.h File Reference	13
4.2 constantes.h	13
4.3 graphiques.c File Reference	13
4.3.1 Detailed Description	14
4.3.2 Function Documentation	14
4.3.2.1 apply_background()	14
4.3.2.2 apply_sprite()	15
4.3.2.3 apply_walls()	15
4.3.2.4 clean()	15
4.3.2.5 clean_textures()	16
4.3.2.6 if_apply_walls()	16
4.3.2.7 init()	16
4.3.2.8 init_textures()	17
4.3.2.9 refresh_graphics()	17
4.4 graphiques.h File Reference	17
4.4.1 Detailed Description	18
4.4.2 Function Documentation	18
4.4.2.1 apply_background()	18
4.4.2.2 apply_sprite()	19
4.4.2.3 apply_walls()	19
4.4.2.4 clean()	19
4.4.2.5 clean_textures()	20
4.4.2.6 if_apply_walls()	20
4.4.2.7 init()	20
4.4.2.8 init_textures()	21
4.4.2.9 refresh_graphics()	21
4.5 graphiques.h	21
4.6 main.c File Reference	22
4.6.1 Detailed Description	22
4.7 sdl2-light.c File Reference	23
4.7.1 Detailed Description	23
4.7.2 Function Documentation	24
4.7.2.1 apply_texture()	24
4.7.2.2 clean_chunk()	24
4.7.2.3 clean_music()	24

4.7.2.4 clean_sdl()	24
4.7.2.5 clean_texture()	26
4.7.2.6 clear_renderer()	26
4.7.2.7 init_sdl()	26
4.7.2.8 load_image()	27
4.7.2.9 pause()	27
4.7.2.10 update_screen()	27
4.8 sdl2-light.h File Reference	28
4.8.1 Detailed Description	28
4.8.2 Function Documentation	29
4.8.2.1 apply_texture()	29
4.8.2.2 clean_chunk()	29
4.8.2.3 clean_music()	29
4.8.2.4 clean_sdl()	29
4.8.2.5 clean_texture()	31
4.8.2.6 clear_renderer()	31
4.8.2.7 init_sdl()	31
4.8.2.8 load_image()	32
4.8.2.9 pause()	32
4.8.2.10 update_screen()	32
4.9 sdl2-light.h	33
4.10 sdl2-ttf-light.c File Reference	33
4.10.1 Detailed Description	34
4.10.2 Function Documentation	34
4.10.2.1 apply_text()	34
4.10.2.2 clean_font()	34
4.10.2.3 load_font()	35
4.11 sdl2-ttf-light.h File Reference	35
4.11.1 Detailed Description	36
4.11.2 Function Documentation	36
4.11.2.1 apply_text()	36
4.11.2.2 clean_font()	36
4.11.2.3 load_font()	37
4.12 sdl2-ttf-light.h	37
4.13 world.c File Reference	37
4.13.1 Detailed Description	38
4.13.2 Function Documentation	38
4.13.2.1 clean_data()	38
4.13.2.2 handle_events()	39
4.13.2.3 handle_sprites_collision()	39
4.13.2.4 init_data()	39
4.13.2.5 init_sprite()	39

4.13.2.6 init_walls()	40
4.13.2.7 is_game_over()	40
4.13.2.8 print_sprint()	40
4.13.2.9 sprites_collide()	41
4.13.2.10 tir_missile()	41
4.13.2.11 update_data()	41
4.13.2.12 update_walls()	41
4.13.2.13 verif_marge()	42
4.14 world.h File Reference	42
4.14.1 Detailed Description	43
4.14.2 Function Documentation	43
4.14.2.1 clean_data()	43
4.14.2.2 handle_events()	44
4.14.2.3 handle_sprites_collision()	44
4.14.2.4 init_data()	44
4.14.2.5 init_sprite()	44
4.14.2.6 init_walls()	45
4.14.2.7 is_game_over()	45
4.14.2.8 print_sprint()	45
4.14.2.9 sprites_collide()	46
4.14.2.10 tir_missile()	46
4.14.2.11 update_data()	46
4.14.2.12 update_walls()	46
4.14.2.13 verif_marge()	47
4.15 world.h	47
Index	49

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

sprite_s	Coordonnés d'un élément du monde du jeu	5
textures_s	Représentation pour stocker les textures nécessaires à l'affichage graphique	6
world_s	Représentation du monde du jeu	9

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

constantes.h	En-tête des constantes utilisés dans le monde du jeu	13
graphiques.c	Partie graphique des données du monde	13
graphiques.h	En-tête de la partie graphique des données du monde	17
main.c	Programme principal initial du niveau 1	22
sdl2-light.c	Sur-couche de SDL2 pour simplifier son utilisation pour le projet	23
sdl2-light.h	En-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet	28
sdl2-ttf-light.c	Programme d'affichage	33
sdl2-ttf-light.h	En tete du programme d'affichage	35
world.c	Partie logique des données du monde	37
world.h	En-tête de la partie logique des données du monde	42

Chapter 3

Class Documentation

3.1 sprite_s Struct Reference

Coordonnés d'un élément du monde du jeu.

```
#include <world.h>
```

Public Attributes

- float `x`
- float `y`
- float `h`
- float `w`
- int `make_disappear`
- int `message`
- int `type_message`
- int `choc_missile`

3.1.1 Detailed Description

Coordonnés d'un élément du monde du jeu.

3.1.2 Member Data Documentation

3.1.2.1 `h`

```
float sprite_s::h
```

hauteur du sprite

3.1.2.2 `make_disappear`

```
int sprite_s::make_disappear
```

pour contrôler la disparition du vaisseau

3.1.2.3 message

```
int sprite_s::message
```

pour contrôler l'affichage du message

3.1.2.4 w

```
float sprite_s::w
```

largeur du sprite

3.1.2.5 x

```
float sprite_s::x
```

abscisse du sprite

3.1.2.6 y

```
float sprite_s::y
```

ordonnée du sprite

The documentation for this struct was generated from the following file:

- [world.h](#)

3.2 textures_s Struct Reference

Représentation pour stocker les textures nécessaires à l'affichage graphique.

```
#include <graphiques.h>
```

Public Attributes

- `SDL_Texture *` [background](#)
- `SDL_Texture *` [spaceship](#)
- `SDL_Texture *` [finish](#)
- `SDL_Texture *` [meteorite](#)
- `TTF_Font *` [font](#)
- `SDL_Texture *` [missile](#)
- `SDL_Texture *` [but](#)
- `Mix_Music *` [music](#)
- `Mix_Chunk *` [sound_win](#)
- `Mix_Chunk *` [sound_lose](#)
- `Mix_Chunk *` [sound_explosion](#)
- `Mix_Chunk *` [sound_mini_explosion](#)
- `Mix_Chunk *` [sound_mini_win](#)

3.2.1 Detailed Description

Représentation pour stocker les textures nécessaires à l'affichage graphique.

3.2.2 Member Data Documentation

3.2.2.1 background

```
SDL_Texture* textures_s::background
```

Texture liée à l'image du fond de l'écran.

3.2.2.2 but

```
SDL_Texture* textures_s::but
```

Texture liée à l'image du but.

3.2.2.3 finish

```
SDL_Texture* textures_s::finish
```

Texture liée à l'image de la ligne d'arrivée.

3.2.2.4 font

```
TTF_Font* textures_s::font
```

Texture liée à l'affichage du texte sur l'écran.

3.2.2.5 meteorite

```
SDL_Texture* textures_s::meteorite
```

Texture liée à l'image de la météorite.

3.2.2.6 missile

```
SDL_Texture* textures_s::missile
```

Texture liée à l'image du missile.

3.2.2.7 music

```
Mix_Music* textures_s::music
```

Texture liée à la musique de fond

3.2.2.8 sound_explosion

```
Mix_Chunk* textures_s::sound_explosion
```

Texture liée à la musique d'explosion ou collision entre mur et vaisseau.

3.2.2.9 sound_lose

```
Mix_Chunk* textures_s::sound_lose
```

Texture liée à la musique de défaite.

3.2.2.10 sound_mini_explosion

```
Mix_Chunk* textures_s::sound_mini_explosion
```

Texture liée à la musique de la petite explosion ou collision entre missile et mur .

3.2.2.11 sound_mini_win

```
Mix_Chunk* textures_s::sound_mini_win
```

Texture liée à la musique de la petite victoire ou collision entre but et vaisseau.

3.2.2.12 sound_win

```
Mix_Chunk* textures_s::sound_win
```

Texture liée à la musique de la victoire.

3.2.2.13 spaceship

```
SDL_Texture* textures_s::spaceship
```

Texture liée à l'image du vaisseau.

The documentation for this struct was generated from the following file:

- [graphiques.h](#)

3.3 world_s Struct Reference

Représentation du monde du jeu.

```
#include <world.h>
```

Collaboration diagram for world_s:

Public Attributes

- [sprite_t](#) `sprite`
- [sprite_t](#) `finish_line`
- [sprite_t](#) `missile`
- [sprite_t](#) `but`
- [sprite_t](#) `walls` [30]
- float `vy`
- float `debut_temps`
- float `temps_final`
- char `message_final` [150]
- int `gameover`
- int `stop`
- int `compteur_missile_max`
- int `collision_but`

3.3.1 Detailed Description

Représentation du monde du jeu.

3.3.2 Member Data Documentation

3.3.2.1 but

```
sprite_t world_s::but
```

Champ indiquant la structure des coordonnées du but

3.3.2.2 collision_but

```
int world_s::collision_but
```

Champ indiquant s'il y a collision avec le but

3.3.2.3 compteur_missile_max

```
int world_s::compteur_missile_max
```

Champ indiquant le nombre de missile

3.3.2.4 debut_temps

```
float world_s::debut_temps
```

Champ indiquant le temps au lancement du jeu

3.3.2.5 finish_line

```
sprite_t world_s::finish_line
```

Champ indiquant la structure des coordonnées de la ligne d'arrivée

3.3.2.6 gameover

```
int world_s::gameover
```

Champ indiquant si l'on est à la fin du jeu

3.3.2.7 message_final

```
char world_s::message_final[150]
```

Champ où sera stocké le message final

3.3.2.8 missile

```
sprite_t world_s::missile
```

Champ indiquant la structure des coordonnées du missile

3.3.2.9 sprite

```
sprite_t world_s::sprite
```

Champ indiquant la structure des coordonnées du vaisseau

3.3.2.10 stop

```
int world_s::stop
```

Champ indiquant si l'on est en pause

3.3.2.11 temps_final

```
float world_s::temps_final
```

Champ indiquant le temps a la fin du jeu

3.3.2.12 vy

```
float world_s::vy
```

Champ indiquant la vitesse de déplacement de la ligne d'arrivée et du mur de météorites

3.3.2.13 walls

```
sprite_t world_s::walls[30]
```

Champ indiquant l'ensemble des murs du jeu

The documentation for this struct was generated from the following file:

- [world.h](#)

Chapter 4

File Documentation

4.1 constantes.h File Reference

en-tête des constantes utilisés dans le monde du jeu

This graph shows which files directly or indirectly include this file:

4.2 constantes.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef CONSTANTES_H
00010 #define CONSTANTES_H
00011
00015 #define SCREEN_WIDTH 300
00016
00020 #define SCREEN_HEIGHT 480
00021
00025 #define SHIP_SIZE 32
00026
00030 #define METEORITE_SIZE 32
00031
00035 #define FINISH_LINE_HEIGHT 10
00036
00040 #define MOVING_STEP 10
00041
00045 #define INITIAL_SPEED 2.0
00046
00050 #define PAS_VITESSE 0.2
00051
00052
00053
00054 #endif
```

4.3 graphiques.c File Reference

partie graphique des données du monde

```
#include "graphiques.h"
#include <stdio.h>
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
#include <SDL2/SDL_mixer.h>
Include dependency graph for graphiques.c:
```

Functions

- void `clean_textures` (`ressources` *textures)
La fonction nettoie les textures.
- void `init_textures` (SDL_Renderer *renderer, `ressources` *textures)
La fonction initialise les textures nécessaires à l'affichage graphique du jeu.
- void `apply_background` (SDL_Renderer *renderer, SDL_Texture *texture)
La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.
- void `apply_sprite` (SDL_Renderer *renderer, SDL_Texture *texture, `sprite_t` *sprite)
La fonction applique la texture du fond sur le renderer lié au sprite.
- void `apply_walls` (SDL_Renderer *renderer, SDL_Texture *texture, `sprite_t` *sprite)
La fonction applique la texture des murs sur le renderer lié au jeu.
- void `if_apply_walls` (SDL_Renderer *renderer, SDL_Texture *texture, `world_t` *world)
La fonction applique la texture des murs sur le renderer lié au jeu si ce mur ne cogne pas un missile.
- void `refresh_graphics` (SDL_Renderer *renderer, `world_t` *world, `ressources` *textures)
La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void `init` (SDL_Window **window, SDL_Renderer **renderer, `ressources` *textures, `world_t` *world)
fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données
- void `clean` (SDL_Window *window, SDL_Renderer *renderer, `ressources` *textures, `world_t` *world)
fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

4.3.1 Detailed Description

partie graphique des données du monde

Author

Jules Hinson / Ahoomey-Zunu Nandi

Version

0.2

Date

5 avril 2025

4.3.2 Function Documentation

4.3.2.1 `apply_background()`

```
void apply_background (
    SDL_Renderer * renderer,
    SDL_Texture * texture )
```

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond

4.3.2.2 apply_sprite()

```
void apply_sprite (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    sprite_t * sprite )
```

La fonction applique la texture du fond sur le renderer lié au sprite.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>sprite</i>	un élément du monde du jeu

4.3.2.3 apply_walls()

```
void apply_walls (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    sprite_t * sprite )
```

La fonction applique la texture des murs sur le renderer lié au jeu.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>sprite</i>	un mur

4.3.2.4 clean()

```
void clean (
    SDL_Window * window,
    SDL_Renderer * renderer,
    ressources * textures,
    world_t * world )
```

fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

Parameters

<i>window</i>	la fenêtre du jeu
---------------	-------------------

Parameters

<i>renderer</i>	le renderer
<i>textures</i>	les textures
<i>world</i>	le monde

4.3.2.5 clean_textures()

```
void clean_textures (
    ressources * textures )
```

La fonction nettoie les textures.

Parameters

<i>textures</i>	les textures
-----------------	--------------

4.3.2.6 if_apply_walls()

```
void if_apply_walls (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    world_t * world )
```

La fonction applique la texture des murs sur le renderer lié au jeu si ce mur ne cogne pas un missile.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>world</i>	le monde

4.3.2.7 init()

```
void init (
    SDL_Window ** window,
    SDL_Renderer ** renderer,
    ressources * textures,
    world_t * world )
```

fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données

Parameters

<i>window</i>	la fenêtre du jeu
<i>renderer</i>	le renderer
<i>textures</i>	les textures
<i>world</i>	le monde

4.3.2.8 init_textures()

```
void init_textures (
    SDL_Renderer * renderer,
    ressources * textures )
```

La fonction initialise les textures nécessaires à l'affichage graphique du jeu.

Parameters

<i>renderer</i>	la surface correspondant à l'écran de jeu
<i>textures</i>	les textures du jeu

4.3.2.9 refresh_graphics()

```
void refresh_graphics (
    SDL_Renderer * renderer,
    world_t * world,
    ressources * textures )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

Parameters

<i>renderer</i>	le renderer lié à l'écran de jeu
<i>world</i>	les données du monde
<i>textures</i>	les textures

4.4 graphiques.h File Reference

en-tête de la partie graphique des données du monde

```
#include "constantes.h"
#include "world.h"
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
#include <SDL2/SDL_mixer.h>
```

Include dependency graph for graphiques.h: This graph shows which files directly or indirectly include this file:

Classes

- struct [textures_s](#)

Représentation pour stocker les textures nécessaires à l'affichage graphique.

Typedefs

- typedef struct [textures_s](#) **ressources**

Type qui correspond aux textures du jeu.

Functions

- void `clean_textures` (`ressources` *textures)
La fonction nettoie les textures.
- void `init_textures` (SDL_Renderer *renderer, `ressources` *textures)
La fonction initialise les textures nécessaires à l'affichage graphique du jeu.
- void `apply_background` (SDL_Renderer *renderer, SDL_Texture *texture)
La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.
- void `apply_sprite` (SDL_Renderer *renderer, SDL_Texture *texture, `sprite_t` *sprite)
La fonction applique la texture du fond sur le renderer lié au sprite.
- void `apply_walls` (SDL_Renderer *renderer, SDL_Texture *texture, `sprite_t` *sprite)
La fonction applique la texture des murs sur le renderer lié au jeu.
- void `if_apply_walls` (SDL_Renderer *renderer, SDL_Texture *texture, `world_t` *world)
La fonction applique la texture des murs sur le renderer lié au jeu si ce mur ne cogne pas un missile.
- void `refresh_graphics` (SDL_Renderer *renderer, `world_t` *world, `ressources` *textures)
La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void `init` (SDL_Window **window, SDL_Renderer **renderer, `ressources` *textures, `world_t` *world)
fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données
- void `clean` (SDL_Window *window, SDL_Renderer *renderer, `ressources` *textures, `world_t` *world)
fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

4.4.1 Detailed Description

en-tête de la partie graphique des données du monde

Author

Jules Hinson / Ahoomey-Zunu Nandi

Version

0.2

Date

5 avril 2025

4.4.2 Function Documentation

4.4.2.1 `apply_background()`

```
void apply_background (
    SDL_Renderer * renderer,
    SDL_Texture * texture )
```

La fonction applique la texture du fond sur le renderer lié à l'écran de jeu.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond

4.4.2.2 apply_sprite()

```
void apply_sprite (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    sprite_t * sprite )
```

La fonction applique la texture du fond sur le renderer lié au sprite.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>sprite</i>	un élément du monde du jeu

4.4.2.3 apply_walls()

```
void apply_walls (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    sprite_t * sprite )
```

La fonction applique la texture des murs sur le renderer lié au jeu.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>sprite</i>	un mur

4.4.2.4 clean()

```
void clean (
    SDL_Window * window,
    SDL_Renderer * renderer,
    ressources * textures,
    world_t * world )
```

fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des textures, nettoyage des données

Parameters

<i>window</i>	la fenêtre du jeu
---------------	-------------------

Parameters

<i>renderer</i>	le renderer
<i>textures</i>	les textures
<i>world</i>	le monde

4.4.2.5 clean_textures()

```
void clean_textures (
    ressources * textures )
```

La fonction nettoie les textures.

Parameters

<i>textures</i>	les textures
-----------------	--------------

4.4.2.6 if_apply_walls()

```
void if_apply_walls (
    SDL_Renderer * renderer,
    SDL_Texture * texture,
    world_t * world )
```

La fonction applique la texture des murs sur le renderer lié au jeu si ce mur ne cogne pas un missile.

Parameters

<i>renderer</i>	le renderer
<i>texture</i>	la texture liée au fond
<i>world</i>	le monde

4.4.2.7 init()

```
void init (
    SDL_Window ** window,
    SDL_Renderer ** renderer,
    ressources * textures,
    world_t * world )
```

fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des textures, initialisation des données

Parameters

<i>window</i>	la fenêtre du jeu
<i>renderer</i>	le renderer
<i>textures</i>	les textures
<i>world</i>	le monde

4.4.2.8 init_textures()

```
void init_textures (
    SDL_Renderer * renderer,
    ressources * textures )
```

La fonction initialise les textures nécessaires à l'affichage graphique du jeu.

Parameters

<i>renderer</i>	la surface correspondant à l'écran de jeu
<i>textures</i>	les textures du jeu

4.4.2.9 refresh_graphics()

```
void refresh_graphics (
    SDL_Renderer * renderer,
    world_t * world,
    ressources * textures )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

Parameters

<i>renderer</i>	le renderer lié à l'écran de jeu
<i>world</i>	les données du monde
<i>textures</i>	les textures

4.5 graphiques.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef GRAPHIQUES_H
00010 #define GRAPHIQUES_H
00011 #include "constantes.h"
00012 #include "world.h"
00013 #include "sdl2-light.h"
00014 #include "sdl2-ttf-light.h"
00015 #include <SDL2/SDL_mixer.h>
00016
00017
00021 struct textures_s{
00022
00023     SDL_Texture* background;
00025     SDL_Texture* spaceship;
00027     SDL_Texture* finish;
00029     SDL_Texture* meteorite;
00031     TTF_Font* font;
00033     SDL_Texture* missile;
00035     SDL_Texture* but;
00037     Mix_Music* music;
00039     Mix_Chunk* sound_win;
00041     Mix_Chunk* sound_lose;
00043     Mix_Chunk* sound_explosion;
00045     Mix_Chunk* sound_mini_explosion;
00047     Mix_Chunk* sound_mini_win;
00049 };
00050
00051
```

```

00055 typedef struct textures_s ressources;
00056
00057
00062 void clean_textures(ressources *textures);
00063
00064
00070 void init_textures(SDL_Renderer *renderer, ressources *textures);
00071
00072
00078 void apply_background(SDL_Renderer *renderer, SDL_Texture *texture);
00079
00080
00087 void apply_sprite(SDL_Renderer *renderer, SDL_Texture *texture, sprite_t * sprite);
00088
00089
00096 void apply_walls(SDL_Renderer *renderer, SDL_Texture *texture, sprite_t * sprite);
00097
00098
00105 void if_apply_walls(SDL_Renderer *renderer, SDL_Texture *texture, world_t * world);
00106
00113 void refresh_graphics(SDL_Renderer *renderer, world_t *world, ressources *textures);
00114
00115
00123 void init(SDL_Window **window, SDL_Renderer ** renderer, ressources *textures, world_t * world);
00124
00132 void clean(SDL_Window *window, SDL_Renderer * renderer, ressources *textures, world_t * world);
00133
00134
00135 #endif

```

4.6 main.c File Reference

Programme principal initial du niveau 1.

```

#include "constantes.h"
#include "world.h"
#include "graphiques.h"
#include "sdl2-light.h"
#include <stdio.h>
#include "sdl2-ttf-light.h"

```

Include dependency graph for main.c:

Functions

- int **main** (int argc, char *args[])
programme principal qui implémente la boucle du jeu

4.6.1 Detailed Description

Programme principal initial du niveau 1.

Author

Mathieu Constant

Version

1.0

Date

18 mars 2021

4.7 sdl2-light.c File Reference

sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include "sdl2-light.h"
#include <stdio.h>
#include <stdlib.h>
Include dependency graph for sdl2-light.c:
```

Functions

- int [init_sdl](#) (SDL_Window **window, SDL_Renderer **renderer, int width, int height)
La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- SDL_Texture * [load_image](#) (const char path[], SDL_Renderer *renderer)
La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- void [apply_texture](#) (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)
La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void [clean_texture](#) (SDL_Texture *texture)
La fonction nettoie une texture en mémoire.
- void [clear_renderer](#) (SDL_Renderer *renderer)
La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void [clean_music](#) (Mix_Music *music)
Libère la mémoire associée à la musique.
- void [clean_chunk](#) (Mix_Chunk *chunk)
Libère la mémoire associée à un effet sonore.
- void [update_screen](#) (SDL_Renderer *renderer)
La fonction met à jour l'écran avec le contenu du renderer.
- void [pause](#) (int time)
La fonction met le programme en pause pendant un laps de temps.
- void [clean_sdl](#) (SDL_Renderer *renderer, SDL_Window *window)
La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

4.7.1 Detailed Description

sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.7.2 Function Documentation

4.7.2.1 apply_texture()

```
void apply_texture (
    SDL_Texture * texture,
    SDL_Renderer * renderer,
    int x,
    int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

<i>texture</i>	la texture que l'on va appliquer
<i>renderer</i>	le renderer qui va recevoir la texture
<i>x</i>	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
<i>y</i>	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.7.2.2 clean_chunk()

```
void clean_chunk (
    Mix_Chunk * chunk )
```

Libère la mémoire associée à un effet sonore.

Parameters

<i>chunk</i>	l'effet sonore à libérer
--------------	--------------------------

4.7.2.3 clean_music()

```
void clean_music (
    Mix_Music * music )
```

Libère la mémoire associée à la musique.

Parameters

<i>music</i>	la musique à libérer
--------------	----------------------

4.7.2.4 clean_sdl()

```
void clean_sdl (
    SDL_Renderer * renderer,
    SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

<i>renderer</i>	le renderer à nettoyer
<i>window</i>	la fenêtre à nettoyer

4.7.2.5 clean_texture()

```
void clean_texture (
    SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

<i>texture</i>	la texture à nettoyer
----------------	-----------------------

4.7.2.6 clear_renderer()

```
void clear_renderer (
    SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

<i>renderer</i>	le renderer de l'écran
-----------------	------------------------

4.7.2.7 init_sdl()

```
int init_sdl (
    SDL_Window ** window,
    SDL_Renderer ** renderer,
    int width,
    int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

<i>window</i>	la fenêtre du jeu
<i>renderer</i>	le renderer
<i>width</i>	largeur de l'écran de jeu
<i>height</i>	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.7.2.8 load_image()

```
SDL_Texture * load_image (
    const char path[],
    SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

<i>path</i>	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
<i>renderer</i>	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.7.2.9 pause()

```
void pause (
    int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

<i>time</i>	ce laps de temps en milliseconde
-------------	----------------------------------

4.7.2.10 update_screen()

```
void update_screen (
    SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

<i>renderer</i>	le renderer de l'écran
-----------------	------------------------

4.8 sdl2-light.h File Reference

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_mixer.h>
```

Include dependency graph for sdl2-light.h: This graph shows which files directly or indirectly include this file:

Functions

- void [clean_sdl](#) (SDL_Renderer *renderer, SDL_Window *window)
La fonction nettoie le renderer et la fenêtre du jeu en mémoire.
- SDL_Texture * [load_image](#) (const char path[], SDL_Renderer *renderer)
La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- int [init_sdl](#) (SDL_Window **window, SDL_Renderer **renderer, int width, int height)
La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- void [clean_texture](#) (SDL_Texture *texture)
La fonction nettoie une texture en mémoire.
- void [apply_texture](#) (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)
La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void [clear_renderer](#) (SDL_Renderer *renderer)
La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void [clean_music](#) (Mix_Music *music)
Libère la mémoire associée à la musique.
- void [clean_chunk](#) (Mix_Chunk *chunk)
Libère la mémoire associée à un effet sonore.
- void [update_screen](#) (SDL_Renderer *renderer)
La fonction met à jour l'écran avec le contenu du renderer.
- void [pause](#) (int time)
La fonction met le programme en pause pendant un laps de temps.

4.8.1 Detailed Description

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.8.2 Function Documentation

4.8.2.1 apply_texture()

```
void apply_texture (
    SDL_Texture * texture,
    SDL_Renderer * renderer,
    int x,
    int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

<i>texture</i>	la texture que l'on va appliquer
<i>renderer</i>	le renderer qui va recevoir la texture
<i>x</i>	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
<i>y</i>	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.8.2.2 clean_chunk()

```
void clean_chunk (
    Mix_Chunk * chunk )
```

Libère la mémoire associée à un effet sonore.

Parameters

<i>chunk</i>	l'effet sonore à libérer
--------------	--------------------------

4.8.2.3 clean_music()

```
void clean_music (
    Mix_Music * music )
```

Libère la mémoire associée à la musique.

Parameters

<i>music</i>	la musique à libérer
--------------	----------------------

4.8.2.4 clean_sdl()

```
void clean_sdl (
    SDL_Renderer * renderer,
    SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

<i>renderer</i>	le renderer à nettoyer
<i>window</i>	la fenêtre à nettoyer

4.8.2.5 clean_texture()

```
void clean_texture (
    SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

<i>texture</i>	la texture à nettoyer
----------------	-----------------------

4.8.2.6 clear_renderer()

```
void clear_renderer (
    SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

<i>renderer</i>	le renderer de l'écran
-----------------	------------------------

4.8.2.7 init_sdl()

```
int init_sdl (
    SDL_Window ** window,
    SDL_Renderer ** renderer,
    int width,
    int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

<i>window</i>	la fenêtre du jeu
<i>renderer</i>	le renderer
<i>width</i>	largeur de l'écran de jeu
<i>height</i>	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.8.2.8 load_image()

```
SDL_Texture * load_image (
    const char path[],
    SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

<i>path</i>	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
<i>renderer</i>	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier *path* n'existe pas)

4.8.2.9 pause()

```
void pause (
    int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

<i>time</i>	ce laps de temps en milliseconde
-------------	----------------------------------

4.8.2.10 update_screen()

```
void update_screen (
    SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

<i>renderer</i>	le renderer de l'écran
-----------------	------------------------

4.9 sdl2-light.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef __SDL2_LIGHT_H__
00011 #define __SDL2_LIGHT_H__
00012
00013 #include <SDL2/SDL.h>
00014 #include <SDL2/SDL_mixer.h>
00015
00016
00023 void clean_sdl(SDL_Renderer *renderer, SDL_Window *window);
00024
00025
00034 SDL_Texture *load_image(const char path[], SDL_Renderer *renderer);
00035
00036
00047 int init_sdl(SDL_Window **window, SDL_Renderer **renderer, int width, int height);
00048
00049
00055 void clean_texture(SDL_Texture *texture);
00056
00057
00066 void apply_texture(SDL_Texture *texture, SDL_Renderer *renderer, int x, int y);
00067
00068
00074 void clear_renderer(SDL_Renderer *renderer);
00075
00081 void clean_music(Mix_Music *music);
00082
00088 void clean_chunk(Mix_Chunk *chunk);
00089
00095 void update_screen(SDL_Renderer *renderer);
00096
00097
00103 void pause(int time);
00104
00105
00106 #endif

```

4.10 sdl2-ttf-light.c File Reference

Programme d'affichage.

```

#include "sdl2-ttf-light.h"
#include "graphiques.h"
#include <stdio.h>
#include "sdl2-light.h"

```

Include dependency graph for sdl2-ttf-light.c:

Functions

- void **init_ttf** ()
La fonction initialise l'environnement TTF.
- TTF_Font * **load_font** (const char *path, int font_size)
La fonction charge une police.
- void **apply_text** (SDL_Renderer *renderer, int x, int y, int w, int h, const char *text, TTF_Font *font)
La fonction applique un texte dans une certaine police sur le renderer à une certaine position et avec une certaine dimension.
- void **clean_font** (TTF_Font *font)
La fonction qui nettoie la police.

4.10.1 Detailed Description

Programme d'affichage.

Author

Mathieu Constant

Version

1.0

Date

18 mars 2021

4.10.2 Function Documentation

4.10.2.1 `apply_text()`

```
void apply_text (
    SDL_Renderer * renderer,
    int x,
    int y,
    int w,
    int h,
    const char * text,
    TTF_Font * font )
```

La fonction applique un texte dans une certaine police sur le renderer à une certaine position et avec une certaine dimension.

Parameters

<i>renderer</i>	le renderer
<i>x</i>	abscisse du coin en haut à gauche du texte
<i>y</i>	son abscisse
<i>w</i>	la largeur du message
<i>h</i>	sa hauteur
<i>text</i>	le texte à afficher
<i>font</i>	la police

4.10.2.2 `clean_font()`

```
void clean_font (
    TTF_Font * font )
```

La fonction qui nettoie la police.

Parameters

<i>font</i>	la police
-------------	-----------

4.10.2.3 load_font()

```
TTF_Font * load_font (
    const char * path,
    int font_size )
```

La fonction charge une police.

Parameters

<i>path</i>	le chemin du fichier correspondant à la police
<i>font_size</i>	la taille de la police

Returns

la police chargée

4.11 sdl2-ttf-light.h File Reference

en tete du programme d'affichage

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include "sdl2-light.h"
```

Include dependency graph for sdl2-ttf-light.h: This graph shows which files directly or indirectly include this file:

Functions

- void **init_ttf** ()
La fonction initialise l'environnement TTF.
- TTF_Font * **load_font** (const char *path, int font_size)
La fonction charge une police.
- void **apply_text** (SDL_Renderer *renderer, int x, int y, int w, int h, const char *text, TTF_Font *font)
La fonction applique un texte dans une certaine police sur le renderer à une certaine position et avec une certaine dimension.
- void **clean_font** (TTF_Font *font)
La fonction qui nettoie la police.

4.11.1 Detailed Description

en tete du programme d'affichage

Author

Mathieu Constant

Version

1.0

Date

18 mars 2021

4.11.2 Function Documentation

4.11.2.1 apply_text()

```
void apply_text (
    SDL_Renderer * renderer,
    int x,
    int y,
    int w,
    int h,
    const char * text,
    TTF_Font * font )
```

La fonction applique un texte dans une certaine police sur le renderer à une certaine position et avec une certaine dimension.

Parameters

<i>renderer</i>	le renderer
<i>x</i>	abscisse du coin en haut à gauche du texte
<i>y</i>	son abscisse
<i>w</i>	la largeur du message
<i>h</i>	sa hauteur
<i>text</i>	le texte à afficher
<i>font</i>	la police

4.11.2.2 clean_font()

```
void clean_font (
    TTF_Font * font )
```

La fonction qui nettoie la police.

Parameters

<i>font</i>	la police
-------------	-----------

4.11.2.3 load_font()

```
TTF_Font * load_font (
    const char * path,
    int font_size )
```

La fonction charge une police.

Parameters

<i>path</i>	le chemin du fichier correspondant à la police
<i>font_size</i>	la taille de la police

Returns

la police chargée

4.12 sdl2-ttf-light.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef __SDL2_TTF_LIGHT__H
00010 #define __SDL2_TTF_LIGHT__H
00011
00012 #include <SDL2/SDL.h>
00013 #include <SDL2/SDL_ttf.h>
00014 #include "sdl2-light.h"
00015
00016
00020 void init_ttf();
00021
00028 TTF_Font * load_font(const char* path, int font_size);
00029
00040 void apply_text(SDL_Renderer *renderer,int x, int y, int w, int h, const char *text, TTF_Font *font);
00041
00046 void clean_font(TTF_Font * font);
00047
00048 #endif
```

4.13 world.c File Reference

partie logique des données du monde

```
#include "world.h"
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
#include <stdio.h>
#include <stdlib.h>
Include dependency graph for world.c:
```

Functions

- void `init_sprite` (`sprite_t` *sprite, int x, int y, int w, int h)
La fonction qui initialise les coordonnées des éléments.
- void `print_sprite` (`sprite_t` *sprite)
La fonction qui affiche les positions du sprite.
- void `init_walls` (`world_t` *world)
La fonction initialise les données les murs de météorites.
- void `init_data` (`world_t` *world)
La fonction initialise les données du monde du jeu.
- void `clean_data` (`world_t` *world)
La fonction nettoie les données du monde.
- int `is_game_over` (`world_t` *world)
La fonction indique si le jeu est fini en fonction des données du monde.
- void `verif_marge` (`world_t` *world)
La fonction verifie si on a atteint les marges de l'écran.
- int `sprites_collide` (`sprite_t` *sp1, `sprite_t` *sp2)
La fonction indique si les 2 sprites passés en paramètres sont en collision.
- void `handle_sprites_collision` (`sprite_t` *sp1, `sprite_t` *sp2, `world_t` *world)
La fonction annule la vitesse de déplacement verticale vy si les 2 sprites passés en paramètres sont en collision.
- void `update_walls` (`world_t` *world)
La fonction met à jour les données du mur en tenant compte de la physique du monde.
- void `tir_missile` (`world_t` *world)
Gère le tir du missile et sa collision avec les murs.
- void `update_data` (`world_t` *world)
La fonction met à jour les données en tenant compte de la physique du monde.
- void `handle_events` (`SDL_Event` *event, `world_t` *world)
La fonction gère les événements ayant eu lieu et qui n'ont pas encore été traités.

4.13.1 Detailed Description

partie logique des données du monde

Author

Jules Hinson / Ahoomey-Zunu Nandi

Version

0.2

Date

5 avril 2025

4.13.2 Function Documentation

4.13.2.1 `clean_data()`

```
void clean_data (
    world_t * world )
```

La fonction nettoie les données du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.13.2.2 handle_events()

```
void handle_events (
    SDL_Event * event,
    world_t * world )
```

La fonction gère les évènements ayant eu lieu et qui n'ont pas encore été traités.

Parameters

<i>event</i>	paramètre qui contient les événements
<i>world</i>	les données du monde

4.13.2.3 handle_sprites_collision()

```
void handle_sprites_collision (
    sprite_t * sp1,
    sprite_t * sp2,
    world_t * world )
```

La fonction annule la vitesse de déplacement verticale vy si les 2 sprites passés en paramètres sont en collision.

Parameters

<i>sp1</i>	le premier sprite
<i>sp2</i>	le deuxième sprite
<i>world</i>	le monde

4.13.2.4 init_data()

```
void init_data (
    world_t * world )
```

La fonction initialise les données du monde du jeu.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.13.2.5 init_sprite()

```
void init_sprite (
```

```
    sprite_t * sprite,  
    int x,  
    int y,  
    int w,  
    int h )
```

La fonction qui initialise les coordonnées des éléments.

Parameters

<i>sprite</i>	Un élément du jeu
<i>x</i>	abscisse du sprite
<i>y</i>	ordonnée du sprite
<i>w</i>	largeur du sprite
<i>h</i>	hauteur du sprite

4.13.2.6 init_walls()

```
void init_walls (  
    world_t * world )
```

La fonction initialise les données les murs de météorites.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.13.2.7 is_game_over()

```
int is_game_over (  
    world_t * world )
```

La fonction indique si le jeu est fini en fonction des données du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

Returns

1 si le jeu est fini, 0 sinon

4.13.2.8 print_sprite()

```
void print_sprite (  
    sprite_t * sprite )
```

La fonction qui affiche les positions du sprite.

Parameters

<i>sprite</i>	Un élément du jeu
---------------	-------------------

4.13.2.9 sprites_collide()

```
int sprites_collide (
    sprite_t * sp1,
    sprite_t * sp2 )
```

La fonction indique si les 2 sprites passés en paramètres sont en collision.

Parameters

<i>sp1</i>	le premier sprite
<i>sp2</i>	le deuxième sprite

4.13.2.10 tir_missile()

```
void tir_missile (
    world_t * world )
```

Gère le tir du missile et sa collision avec les murs.

Parameters

<i>world</i>	le monde
--------------	----------

4.13.2.11 update_data()

```
void update_data (
    world_t * world )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.13.2.12 update_walls()

```
void update_walls (
    world_t * world )
```

La fonction met à jour les données du mur en tenant compte de la physique du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.13.2.13 `verif_marge()`

```
void verif_marge (
    world_t * world )
```

La fonction verifie si on a atteint les marges de l'écran.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14 `world.h` File Reference

en-tête de la partie logique des données du monde

```
#include "constantes.h"
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
```

Include dependency graph for `world.h`: This graph shows which files directly or indirectly include this file:

Classes

- struct `sprite_s`
Coordonnés d'un élément du monde du jeu.
- struct `world_s`
Représentation du monde du jeu.

Typedefs

- typedef struct `sprite_s` `sprite_t`
Type qui correspond aux coordonnées des éléments du jeu.
- typedef struct `world_s` `world_t`
Type qui correspond aux données du monde.

Functions

- void `init_sprite` (`sprite_t` *sprite, int x, int y, int w, int h)
La fonction qui initialise les coordonnées des éléments.
- void `print_sprite` (`sprite_t` *sprite)
La fonction qui affiche les positions du sprite.
- void `init_walls` (`world_t` *world)
La fonction initialise les données les murs de météorites.
- void `init_data` (`world_t` *world)
La fonction initialise les données du monde du jeu.
- void `clean_data` (`world_t` *world)
La fonction nettoie les données du monde.
- int `is_game_over` (`world_t` *world)
La fonction indique si le jeu est fini en fonction des données du monde.
- void `verif_marge` (`world_t` *world)
La fonction verifie si on a atteint les marges de l'écran.
- int `sprites_collide` (`sprite_t` *sp1, `sprite_t` *sp2)
La fonction indique si les 2 sprites passés en paramètres sont en collision.
- void `handle_sprites_collision` (`sprite_t` *sp1, `sprite_t` *sp2, `world_t` *world)
La fonction annule la vitesse de déplacement verticale vy si les 2 sprites passés en paramètres sont en collision.
- void `update_walls` (`world_t` *world)
La fonction met à jour les données du mur en tenant compte de la physique du monde.
- void `tir_missile` (`world_t` *world)
Gère le tir du missile et sa collision avec les murs.
- void `update_data` (`world_t` *world)
La fonction met à jour les données en tenant compte de la physique du monde.
- void `handle_events` (`SDL_Event` *event, `world_t` *world)
La fonction gère les événements ayant eu lieu et qui n'ont pas encore été traités.

4.14.1 Detailed Description

en-tête de la partie logique des données du monde

Author

Jules Hinson / Ahoomey-Zunu Nandi

Version

0.2

Date

5 avril 2025

4.14.2 Function Documentation

4.14.2.1 `clean_data()`

```
void clean_data (  
    world_t * world )
```

La fonction nettoie les données du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14.2.2 handle_events()

```
void handle_events (
    SDL_Event * event,
    world_t * world )
```

La fonction gère les évènements ayant eu lieu et qui n'ont pas encore été traités.

Parameters

<i>event</i>	paramètre qui contient les événements
<i>world</i>	les données du monde

4.14.2.3 handle_sprites_collision()

```
void handle_sprites_collision (
    sprite_t * sp1,
    sprite_t * sp2,
    world_t * world )
```

La fonction annule la vitesse de déplacement verticale vy si les 2 sprites passés en paramètres sont en collision.

Parameters

<i>sp1</i>	le premier sprite
<i>sp2</i>	le deuxième sprite
<i>world</i>	le monde

4.14.2.4 init_data()

```
void init_data (
    world_t * world )
```

La fonction initialise les données du monde du jeu.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14.2.5 init_sprite()

```
void init_sprite (
```

```
sprite_t * sprite,  
int x,  
int y,  
int w,  
int h )
```

La fonction qui initialise les coordonnées des éléments.

Parameters

<i>sprite</i>	Un élément du jeu
<i>x</i>	abscisse du sprite
<i>y</i>	ordonnée du sprite
<i>w</i>	largeur du sprite
<i>h</i>	hauteur du sprite

4.14.2.6 init_walls()

```
void init_walls (  
    world_t * world )
```

La fonction initialise les données les murs de météorites.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14.2.7 is_game_over()

```
int is_game_over (  
    world_t * world )
```

La fonction indique si le jeu est fini en fonction des données du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

Returns

1 si le jeu est fini, 0 sinon

4.14.2.8 print_sprite()

```
void print_sprite (  
    sprite_t * sprite )
```

La fonction qui affiche les positions du sprite.

Parameters

<i>sprite</i>	Un élément du jeu
---------------	-------------------

4.14.2.9 sprites_collide()

```
int sprites_collide (
    sprite_t * sp1,
    sprite_t * sp2 )
```

La fonction indique si les 2 sprites passés en paramètres sont en collision.

Parameters

<i>sp1</i>	le premier sprite
<i>sp2</i>	le deuxième sprite

4.14.2.10 tir_missile()

```
void tir_missile (
    world_t * world )
```

Gère le tir du missile et sa collision avec les murs.

Parameters

<i>world</i>	le monde
--------------	----------

4.14.2.11 update_data()

```
void update_data (
    world_t * world )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14.2.12 update_walls()

```
void update_walls (
    world_t * world )
```

La fonction met à jour les données du mur en tenant compte de la physique du monde.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.14.2.13 verif_marge()

```
void verif_marge (
    world_t * world )
```

La fonction verifie si on a atteint les marges de l'écran.

Parameters

<i>world</i>	les données du monde
--------------	----------------------

4.15 world.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef WORLD_H
00010 #define WORLD_H
00011 #include "constantes.h"
00012 #include "sdl2-light.h"
00013 #include "sdl2-ttf-light.h"
00014
00019 struct sprite_s{
00020     float x;
00022     float y;
00024     float h;
00026     float w;
00028     int make_disappear;
00030     int message;
00032     int type_message;
00033
00034     int choc_missile;
00035 };
00036
00040 typedef struct sprite_s sprite_t;
00041
00042
00047 struct world_s{
00048
00049     sprite_t sprite;
00051     sprite_t finish_line;
00053     sprite_t missile;
00055     sprite_t but;
00057     sprite_t walls [30];
00059     float vy;
00061     float debut_temps;
00063     float temps_final;
00065     char message_final[150];
00067     int gameover;
00069     int stop;
00071     int compteur_missile_max;
00073     int collision_but;
00075 };
00076
00080 typedef struct world_s world_t;
00081
00082
00092 void init_sprite(sprite_t * sprite,int x,int y,int w,int h);
00093
00094
00100 void print_sprint(sprite_t* sprite);
00101
00107 void init_walls(world_t * world);
00108
00114 void init_data(world_t * world);
```

```
00115
00116
00122 void clean_data(world_t * world);
00123
00124
00131 int is_game_over(world_t * world);
00132
00133
00139 void verif_marge(world_t * world);
00140
00146 int sprites_collide(sprite_t * sp1, sprite_t *sp2);
00147
00148
00155 void handle_sprites_collision(sprite_t* sp1, sprite_t* sp2, world_t* world);
00156
00161 void update_walls(world_t * world);
00162
00167 void tir_missile(world_t *world);
00168
00173 void update_data(world_t * world);
00174
00180 void handle_events(SDL_Event * event,world_t * world);
00181
00182
00183 #endif
```

Index

- apply_background
 - graphiques.c, [14](#)
 - graphiques.h, [18](#)
- apply_sprite
 - graphiques.c, [15](#)
 - graphiques.h, [19](#)
- apply_text
 - sdl2-ttf-light.c, [34](#)
 - sdl2-ttf-light.h, [36](#)
- apply_texture
 - sdl2-light.c, [24](#)
 - sdl2-light.h, [29](#)
- apply_walls
 - graphiques.c, [15](#)
 - graphiques.h, [19](#)
- background
 - textures_s, [7](#)
- but
 - textures_s, [7](#)
 - world_s, [9](#)
- clean
 - graphiques.c, [15](#)
 - graphiques.h, [19](#)
- clean_chunk
 - sdl2-light.c, [24](#)
 - sdl2-light.h, [29](#)
- clean_data
 - world.c, [38](#)
 - world.h, [43](#)
- clean_font
 - sdl2-ttf-light.c, [34](#)
 - sdl2-ttf-light.h, [36](#)
- clean_music
 - sdl2-light.c, [24](#)
 - sdl2-light.h, [29](#)
- clean_sdl
 - sdl2-light.c, [24](#)
 - sdl2-light.h, [29](#)
- clean_texture
 - sdl2-light.c, [26](#)
 - sdl2-light.h, [31](#)
- clean_textures
 - graphiques.c, [16](#)
 - graphiques.h, [20](#)
- clear_renderer
 - sdl2-light.c, [26](#)
 - sdl2-light.h, [31](#)
- collision_but
 - world_s, [9](#)
- compteur_missile_max
 - world_s, [9](#)
- constantes.h, [13](#)
- debut_temps
 - world_s, [9](#)
- finish
 - textures_s, [7](#)
- finish_line
 - world_s, [10](#)
- font
 - textures_s, [7](#)
- gameover
 - world_s, [10](#)
- graphiques.c, [13](#)
 - apply_background, [14](#)
 - apply_sprite, [15](#)
 - apply_walls, [15](#)
 - clean, [15](#)
 - clean_textures, [16](#)
 - if_apply_walls, [16](#)
 - init, [16](#)
 - init_textures, [17](#)
 - refresh_graphics, [17](#)
- graphiques.h, [17](#)
 - apply_background, [18](#)
 - apply_sprite, [19](#)
 - apply_walls, [19](#)
 - clean, [19](#)
 - clean_textures, [20](#)
 - if_apply_walls, [20](#)
 - init, [20](#)
 - init_textures, [21](#)
 - refresh_graphics, [21](#)
- h
 - sprite_s, [5](#)
- handle_events
 - world.c, [39](#)
 - world.h, [44](#)
- handle_sprites_collision
 - world.c, [39](#)
 - world.h, [44](#)
- if_apply_walls
 - graphiques.c, [16](#)
 - graphiques.h, [20](#)
- init

- graphiques.c, 16
 - graphiques.h, 20
- init_data
 - world.c, 39
 - world.h, 44
- init_sdl
 - sdl2-light.c, 26
 - sdl2-light.h, 31
- init_sprite
 - world.c, 39
 - world.h, 44
- init_textures
 - graphiques.c, 17
 - graphiques.h, 21
- init_walls
 - world.c, 40
 - world.h, 45
- is_game_over
 - world.c, 40
 - world.h, 45
- load_font
 - sdl2-ttf-light.c, 35
 - sdl2-ttf-light.h, 37
- load_image
 - sdl2-light.c, 27
 - sdl2-light.h, 32
- main.c, 22
- make_disappear
 - sprite_s, 5
- message
 - sprite_s, 5
- message_final
 - world_s, 10
- meteorite
 - textures_s, 7
- missile
 - textures_s, 7
 - world_s, 10
- music
 - textures_s, 7
- pause
 - sdl2-light.c, 27
 - sdl2-light.h, 32
- print_sprint
 - world.c, 40
 - world.h, 45
- refresh_graphics
 - graphiques.c, 17
 - graphiques.h, 21
- sdl2-light.c, 23
 - apply_texture, 24
 - clean_chunk, 24
 - clean_music, 24
 - clean_sdl, 24
 - clean_texture, 26
 - clear_renderer, 26
 - init_sdl, 26
 - load_image, 27
 - pause, 27
 - update_screen, 27
- sdl2-light.h, 28
 - apply_texture, 29
 - clean_chunk, 29
 - clean_music, 29
 - clean_sdl, 29
 - clean_texture, 31
 - clear_renderer, 31
 - init_sdl, 31
 - load_image, 32
 - pause, 32
 - update_screen, 32
- sdl2-ttf-light.c, 33
 - apply_text, 34
 - clean_font, 34
 - load_font, 35
- sdl2-ttf-light.h, 35
 - apply_text, 36
 - clean_font, 36
 - load_font, 37
- sound_explosion
 - textures_s, 8
- sound_lose
 - textures_s, 8
- sound_mini_explosion
 - textures_s, 8
- sound_mini_win
 - textures_s, 8
- sound_win
 - textures_s, 8
- spaceship
 - textures_s, 8
- sprite
 - world_s, 10
- sprite_s, 5
 - h, 5
 - make_disappear, 5
 - message, 5
 - w, 6
 - x, 6
 - y, 6
- sprites_collide
 - world.c, 41
 - world.h, 46
- stop
 - world_s, 10
- temps_final
 - world_s, 10
- textures_s, 6
 - background, 7
 - but, 7
 - finish, 7
 - font, 7

- meteorite, [7](#)
- missile, [7](#)
- music, [7](#)
- sound_explosion, [8](#)
- sound_lose, [8](#)
- sound_mini_explosion, [8](#)
- sound_mini_win, [8](#)
- sound_win, [8](#)
- spaceship, [8](#)
- tir_missile
 - world.c, [41](#)
 - world.h, [46](#)
- update_data
 - world.c, [41](#)
 - world.h, [46](#)
- update_screen
 - sdl2-light.c, [27](#)
 - sdl2-light.h, [32](#)
- update_walls
 - world.c, [41](#)
 - world.h, [46](#)
- verif_marge
 - world.c, [42](#)
 - world.h, [47](#)
- vy
 - world_s, [10](#)
- w
 - sprite_s, [6](#)
- walls
 - world_s, [11](#)
- world.c, [37](#)
 - clean_data, [38](#)
 - handle_events, [39](#)
 - handle_sprites_collision, [39](#)
 - init_data, [39](#)
 - init_sprite, [39](#)
 - init_walls, [40](#)
 - is_game_over, [40](#)
 - print_sprint, [40](#)
 - sprites_collide, [41](#)
 - tir_missile, [41](#)
 - update_data, [41](#)
 - update_walls, [41](#)
 - verif_marge, [42](#)
- world.h, [42](#)
 - clean_data, [43](#)
 - handle_events, [44](#)
 - handle_sprites_collision, [44](#)
 - init_data, [44](#)
 - init_sprite, [44](#)
 - init_walls, [45](#)
 - is_game_over, [45](#)
 - print_sprint, [45](#)
 - sprites_collide, [46](#)
 - tir_missile, [46](#)
 - update_data, [46](#)
 - update_walls, [46](#)
 - verif_marge, [47](#)
- world_s, [9](#)
 - but, [9](#)
 - collision_but, [9](#)
 - compteur_missile_max, [9](#)
 - debut_temps, [9](#)
 - finish_line, [10](#)
 - gameover, [10](#)
 - message_final, [10](#)
 - missile, [10](#)
 - sprite, [10](#)
 - stop, [10](#)
 - temps_final, [10](#)
 - vy, [10](#)
 - walls, [11](#)
- x
 - sprite_s, [6](#)
- y
 - sprite_s, [6](#)